

OptiCrop: Smart Agricultural Production Optimization Engine

Project Description:

OptiCrop is a Flask-based crop recommendation platform that helps farmers and agricultural planners identify the most suitable crop for cultivation based on soil composition and environmental conditions. The platform includes a Crop Recommendation Engine, which analyzes seven soil and climate parameters to predict the optimal crop, a Confidence & Probability module, which reports the model's certainty and the likelihood of alternative crops, and a Reporting module, which generates downloadable PDF and CSV summaries of past predictions.

Utilizing a trained scikit-learn / XGBoost classification pipeline, OptiCrop processes Nitrogen, Phosphorus, Potassium, temperature, humidity, pH, and rainfall values to deliver fast, data-driven crop recommendations. Built with Flask and backed by a SQLite database via SQLAlchemy, the platform stores every prediction, powers a metrics dashboard, and ensures a seamless, farmer-friendly experience. With a responsive Bootstrap interface, prediction history, and exportable reports, OptiCrop empowers farmers, agronomists, and agri-tech businesses to make confident, evidence-based planting decisions.

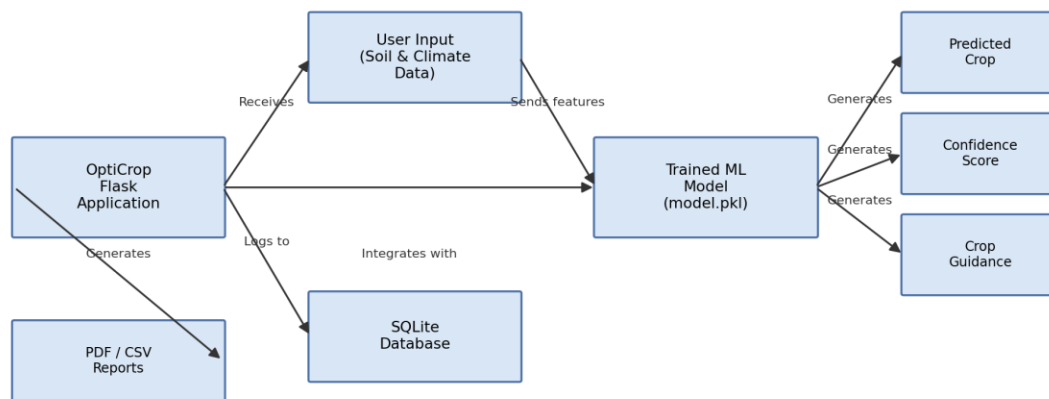
Scenarios

Scenario 1: A farmer in a high-rainfall region enters soil test values for Nitrogen, Phosphorus, and Potassium along with local temperature, humidity, pH, and rainfall readings. OptiCrop instantly recommends rice as the optimal crop, along with a confidence score, the expected growing season, water requirements, and fertilizer guidance.

Scenario 2: An agricultural extension officer wants to compare recommendations across several plots. Using the `/api/predict` endpoint, the officer submits soil parameters programmatically and receives a JSON response with the predicted crop, confidence percentage, and probability breakdown across all twenty-two supported crops for integration into a larger advisory tool.

Scenario 3: A planner reviews the Dashboard to understand recent recommendation trends, checks the History page to audit past predictions, and downloads a PDF report for a specific prediction to share with a cooperative society as supporting documentation for a crop-planning decision.

Technical Architecture:



Description: OptiCrop utilizes a modular architecture to deliver fast, ML-driven crop recommendations. The frontend provides a responsive Bootstrap 5 interface built with HTML, CSS, and JavaScript, while the Flask-based backend validates incoming soil and climate data and routes it to the trained machine-learning pipeline. The model layer (model.py) loads the persisted model, scaler, and label encoder artifacts to generate a prediction, confidence score, and crop-specific guidance, while the database layer (database.py) persists every prediction, report, and dataset/model record in SQLite via SQLAlchemy.

(Note: Since this project uses a database for storing users, predictions, reports, datasets, and ML model metadata, an ER Diagram is included in the repository under docs/er_diagram.md, describing the User, Prediction, Report, Dataset, and MLModel entities and their relationships.)

Pre-requisites:

- **Flask Framework Knowledge:** [Flask Documentation](#)
- **Machine Learning with scikit-learn:** [scikit-learn Documentation](#)
- **Data Handling with Pandas & NumPy:** [Pandas Documentation](#)
- **HTML, CSS, and JavaScript Skills:** [W3Schools HTML/CSS/JavaScript Tutorials](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Database Modeling with SQLAlchemy:** [SQLAlchemy Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Deployment on Render:** [Render Documentation](#)

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Collect and prepare the crop recommendation dataset (2,200 records across 22 crops).
- **Activity 1.2:** Research and select the appropriate classification model for crop prediction.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, backend, ML pipeline, and database.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for Flask, scikit-learn, and SQLAlchemy.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: dataset preprocessing, multi-model training and evaluation, and best-model selection.
- **Activity 2.2:** Implement the Flask backend to manage routing, request validation, and prediction logging to SQLite.

Milestone 3: App.py Development

- **Activity 3.1:** Write the main application logic in app.py, establishing routes for prediction, history, dashboard, and reporting.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML, CSS (Bootstrap 5), and JavaScript, ensuring a responsive and intuitive layout.
- **Activity 4.2:** Create dynamic templates with Flask's render_template to display prediction results, history, and dashboard metrics.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Deploy the application publicly on Render using the Procfile and render.yaml configuration.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, we focus on preparing the dataset and selecting the appropriate machine-learning approach for crop recommendation. This involves cleaning the raw dataset, evaluating multiple candidate classifiers, and ensuring the chosen model aligns with the application's objective of recommending the best crop from soil and climate inputs.

Activity 1.1: Prepare the Dataset

The `crop_recommendation.csv` dataset (2,200 rows) contains seven numeric features - Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall - along with a label column identifying the recommended crop across 22 classes such as rice, maize, chickpea, and cotton.

```
def load_dataset(path: Path = DATASET_PATH) -> pd.DataFrame:
    df = pd.read_csv(path)
    return df

def clean_dataset(df: pd.DataFrame) -> pd.DataFrame:
    df = df.drop_duplicates().dropna().reset_index(drop=True)
    return df
```

Activity 1.2: Research and Select the Appropriate Model

- Understand the Project Requirements: Review the specific needs of OptiCrop - recommending a single best crop from seven numeric soil and climate features.
- Explore Candidate Algorithms: Evaluate Logistic Regression, Decision Tree, Random Forest, KNN, SVM, Naive Bayes, Gradient Boosting, and XGBoost.
- Evaluate Model Performance: Compare models on accuracy, precision, recall, and F1-score using an 80/20 stratified train-test split.
- Select the Optimal Model: The training pipeline automatically selects the classifier with the highest test accuracy and persists it as `model.pkl`, together with the fitted `StandardScaler` and `LabelEncoder`.

```
def build_models() -> Dict[str, Any]:
    models = {
        "Logistic Regression": LogisticRegression(max_iter=1000),
        "Decision Tree": DecisionTreeClassifier(random_state=42),
        "Random Forest": RandomForestClassifier(n_estimators=120, random_state=42),
        "KNN": KNeighborsClassifier(n_neighbors=7),
        "SVM": CalibratedClassifierCV(SVC(kernel="rbf"), cv=3),
        "Naive Bayes": GaussianNB(),
        "Gradient Boosting": GradientBoostingClassifier(random_state=42),
    }
    if XGBOOST_AVAILABLE:
        models["XGBoost"] = XGBClassifier(eval_metric="mlogloss", random_state=42)
    return models
```

Activity 1.3: Define the Architecture of the Application

- Draft an Architectural Diagram: Create a visual representation of the application architecture, including the frontend (HTML, CSS, JavaScript), Flask backend, ML pipeline, and SQLite database.

- **Detail Frontend Functionality:** Outline how users enter soil nutrient values (N, P, K) and climate readings (temperature, humidity, pH, rainfall) on the homepage.
- **Outline Backend Responsibilities:** Specify how the Flask backend handles incoming POST requests to /predict, validates numeric input, scales the feature vector, and queries the trained model.
- **Describe Data Persistence:** Define how every prediction is logged to SQLite via database.py for later use in the History and Dashboard pages.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.9+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv venv
D:\projects>venv\Scripts\activate
(venv) D:\projects>pip install -r requirements.txt
```

- **Key Dependencies (requirements.txt):** Flask, pandas, numpy, scikit-learn, matplotlib, seaborn, joblib, SQLAlchemy, fpdf, gunicorn, python-dotenv, xgboost.
- **Configure Environment Variables:** Create a .env file in the project root (based on .env.example) to store the Flask SECRET_KEY.
- **Set Up Application Structure:** The project directory includes app.py, model.py, database.py, requirements.txt, static/, templates/, dataset/, notebooks/, and docs/.

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

Implement the core machine-learning pipeline that trains, evaluates, and selects the best crop-recommendation model, then exposes it for real-time predictions.

- **Model Training:** `train_and_save()` cleans the dataset, scales features with `StandardScaler`, encodes crop labels with `LabelEncoder`, trains all candidate models, and persists the best-performing model.
- **Prediction Engine:** `predict_crop()` scales a new feature vector, runs it through the trained model, and returns the predicted crop, confidence score, and full probability distribution across all 22 crops.
- **Crop Context:** `get_crop_context()` returns crop-specific guidance - growing season, water requirement, and fertilizer suggestion - for the recommended crop.

```
def predict_crop(model, scaler, encoder, raw_features):
    features = build_feature_vector(raw_features)
    scaled = scaler.transform(features)
    proba = model.predict_proba(scaled)[0]
    prediction_idx = int(model.predict(scaled)[0])
    crop_label = encoder.inverse_transform([prediction_idx])[0]
    confidence = float(max(proba))
    proba_map = {encoder.inverse_transform([i])[0]: float(v)
                 for i, v in enumerate(proba)}
    return crop_label, confidence, proba_map
```

Activity 2.2: Implement the Flask Backend

Define Routes in Flask:

- Set up routes in `app.py` for the homepage, prediction endpoint, history, dashboard, reporting, and REST API.

Process User Input:

- Create a form in `index.html` for users to submit N, P, K, temperature, humidity, pH, and rainfall values.
- Use Flask's `request.form` / `request.get_json()` to retrieve submitted data from the browser or API clients.
- Validate that all seven fields are present, numeric, and non-negative before processing.

Integrate the ML Pipeline:

- Within the `/predict` route handler, call `predict_crop()` with the loaded model, scaler, and encoder.
- Persist each prediction to SQLite using `log_prediction()`, associating it with a guest or registered user.
- Return either a rendered `result.html` page or a structured JSON response, depending on the request type.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of app.py, which serves as the backbone of the OptiCrop application. In this milestone, each route is implemented to handle a distinct feature, from prediction to reporting.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py:

- / - Home page route that renders the input form and recent model metrics.

```
@app.route("/")
def index():
    crop_samples = ["rice", "maize", "lentil", "watermelon"]
    return render_template(
        "index.html",
        title="OptiCrop | Smart Agricultural Production Optimization",
        crop_samples=crop_samples,
        model_metrics=model_metrics,
    )
```

- /predict - POST endpoint that accepts form or JSON input, invokes the ML model, and returns a recommendation.

```
@app.route("/predict", methods=["GET", "POST"])
def predict():
    form_data = request.form or request.get_json(silent=True) or {}
    required_fields = ["n", "p", "k", "temperature", "humidity", "ph", "rainfall"]
    features = [float(form_data.get(f, "")) for f in required_fields]

    crop_name, confidence, probabilities = predict_crop(
        model, scaler, encoder, features)
    crop_info = get_crop_context(crop_name)

    prediction_record = log_prediction(
        db, user.user_id, features, crop_name, confidence, probabilities)

    return render_template(
        "result.html", prediction=prediction_record,
        crop_info=crop_info, probabilities=probabilities,
        rounded_confidence=round(confidence * 100, 2))
```

- /history, /dashboard, /download - Support endpoints for reviewing past predictions, model metrics, and downloading PDF reports.
- /api/health, /api/history, /api/predict - REST-style endpoints for programmatic access and third-party integration.

Set Up Route Handlers for Each Feature:

- For the /predict route, implement logic that reads N, P, K, temperature, humidity, pH, and rainfall from the incoming form or JSON body.
- Apply input validation: redirect with a flash message if any field is missing, non-numeric, or negative.

- Route both browser form submissions and AJAX/API requests to the same /predict logic, branching the response format on request.is_json.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a clean, farmer-friendly interface for OptiCrop. This involves designing a responsive Bootstrap 5 layout and creating dynamic templates that present predictions, history, and dashboard metrics clearly.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop a base.html layout shared across all pages, including a navigation bar, footer, and flash-message region.
- Build index.html as the primary input form, result.html for recommendation output, history.html for past predictions, and dashboard.html for model metrics.
- Use semantic HTML elements and Bootstrap 5 components to keep the structure organized and accessible.

Design a Responsive Layout Using CSS:

- Create a stylesheet (static/css/styles.css) implementing a clean, agriculture-themed aesthetic with card-based sections.
- Use Bootstrap's grid system to organize the input form, results, and dashboard charts responsively across desktop, tablet, and mobile screens.

Create Separate UI Components for Each Output Type:

- Result Card: displays the recommended crop, confidence percentage, growing season, water needs, and fertilizer guidance.
- Probability Chart: visualizes the model's confidence across the top candidate crops.
- History Table: lists past predictions with a CSV export option and PDF download link per record.

Activity 4.2: Creating Dynamic Content Rendering

Handle Form Submission:

- The input form on index.html submits soil and climate values to /predict, which renders result.html with the AI-generated recommendation.

Render Results Dynamically:

- Flask's render_template injects prediction, crop_info, and probabilities into result.html for display.
- dashboard.html aggregates recent predictions and model comparison metrics (accuracy, precision, recall, F1-score) for the selected model.

Support History and Reporting:

- history.html lists all logged predictions and supports a CSV export via the ?format=csv query parameter.
- The /download route generates a per-prediction PDF report using FPDF, summarizing inputs, the recommended crop, and cultivation guidance.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying the OptiCrop application first locally to verify correct operation, and then publicly via Render to make the recommendation engine accessible online.

Activity 5.1: Preparing the Application for Local Deployment

- Create and activate a virtual environment, then install all required dependencies from requirements.txt:

```
D:\projects>python -m venv venv
D:\projects>venv\Scripts\activate
(venv) D:\projects>pip install -r requirements.txt
```

- Copy .env.example to .env and update the SECRET_KEY as needed.

Activity 5.2: Local Testing and Verification

- Start the Flask development server:

```
(venv) D:\projects>python app.py
```

- On first run, ensure_artifacts() automatically trains and saves the model if model.pkl, scaler.pkl, or label_encoder.pkl are missing.
- Open a browser and navigate to the local address printed in the terminal to access the app, then test predictions across a range of soil and climate values.

Activity 5.3: Public Deployment via Render

Render hosts the Flask application on a managed web service, exposing it over a public HTTPS URL without manual server configuration.

- The Procfile defines the production start command using Gunicorn:

```
web: gunicorn app:app
```

- The render.yaml file defines the Render web service configuration:

```
services:
- type: web
  name: opticrops-web
  env: python
  plan: free
  buildCommand: pip install -r requirements.txt
  startCommand: gunicorn app:app
```

- Push the repository to GitHub, connect it to Render, and Render automatically builds and deploys the service using the configuration above.

Important Notes:

- Model artifacts (model.pkl, scaler.pkl, label_encoder.pkl) are generated automatically on first startup if not already present, so no manual training step is required before deployment.

- SQLite is file-based; on Render's free tier the database resets on redeploy unless a persistent disk or external database is configured.
- For a custom domain or always-on availability, upgrade to a paid Render plan.

Exploring the Website's Web Pages:

Home / Input Page:

The homepage presents a clean card-based form where users enter Nitrogen, Phosphorus, Potassium, temperature, humidity, pH, and rainfall values. Sample crop chips (rice, maize, lentil, watermelon) illustrate typical inputs, and a summary of the currently selected model's accuracy is shown alongside the form.

Result Page:

After submission, the result page displays the recommended crop with its confidence percentage, the growing season, water requirement, and fertilizer guidance, along with a breakdown of prediction probabilities across the other candidate crops. A button allows the user to download a PDF report of the recommendation.

Dashboard Page:

The dashboard summarizes recent prediction activity - a count of predictions per crop - alongside model comparison metrics such as accuracy, precision, recall, and F1-score for the model family evaluated during training, and the dataset shape used for training.

History Page:

The history page lists every logged prediction with its input parameters, predicted crop, and confidence, and supports exporting the full history as a CSV file for offline analysis or record-keeping.

About & Contact Pages:

The About page introduces the OptiCrop project and its goals for supporting data-driven agricultural planning, while the Contact page provides a simple way for users to reach the project team with questions or feedback.

Conclusion:

OptiCrop is a machine-learning-powered crop recommendation platform built with Flask and backed by a SQLite database via SQLAlchemy. It evaluates multiple classification algorithms - including Random Forest, Gradient Boosting, and XGBoost - on a 2,200-record agricultural dataset to automatically select the best-performing model for recommending crops from soil nutrient and climate readings. The project, which included dataset preparation, model training and evaluation, backend and frontend development, and deployment via Render, demonstrates how targeted machine learning and a structured Flask architecture can support real-world agricultural decision-making. Looking ahead, OptiCrop offers significant opportunities for expansion, such as integrating live weather-API forecasting, multi-language support for regional farmers, mobile application access, and user authentication to maintain farmer-specific prediction histories. By continuously refining the underlying model and enhancing frontend accessibility, the platform is well-positioned to evolve from a recommendation tool into a comprehensive agricultural advisory assistant.